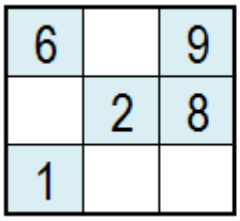
Task 5. Hidato

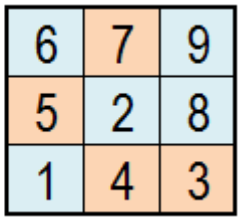
Available marks: 28

The *Hidato* puzzle is often considered a variant of Sudoku, but was invented independently by Israeli computer scientist Gyora Benedek. It can be played on a regular or irregular grid containing n cells. The aim of the game is to place the sequence $1, 2, 3, ..., n–1, n$ in the cells such that:

- cells containing consecutive numbers are touching (connected horizontally, vertically or diagonally), and
- like Sudoku, some of the numbers have already been placed on the grid, and the resulting constraints make the solution unique.

Example Puzzle and Solution





While an exhaustive search will eventually produce a solution, you are unlikely to get full marks with an entirely brute-force approach. There is a strict 5-second real time limit on the completion of any test, so you will have to incorporate some logic to limit the search space.

Your Task

Write a program that accepts a square Hidato puzzle grid and completes it within the 5-second time limit. The first line of input contains two positive integers, separated by a space: the size of the grid and the number of available cells (that is, filled or empty cells, excluding those containing a "+" character as discussed below).

The remaining lines show each row of the grid, with cell values separated by a space. Each cell contains either

- A number,
- A "–" character, which represents an empty cell, or
- A "+" character, which represents an unfillable cell such as a border, allowing irregular shapes to be constructed.

The example would be represented by the file

```
3 9
6 - 9
- 2 8
1 - -
```

If the grid can be solved, show the solution neatly with cells aligned. If the grid cannot be solved, the program should detect and report this. If any filled cells violate the adjacency rule, the program should identify the number that is wrongly placed. Hence it should be able to validate any completed grid, even if it can't produce solutions.

Assessment

As well as the above example (`task5sample.txt`, used for development only), there are 5 assessable test files of (roughly) increasing complexity, named `solve1.txt`, `solve2.txt`, `solve3.txt`, `solve4.txt` and `solve5.txt`. Each one solved within the time limit is worth 4 marks.

There are also 5 completed grids in `check1.txt`, `check2.txt`, `check3.txt`, `check4.txt` and `check5.txt`. Most are invalid. Correctly identifying the errors (if any) in the examples selected by the judges is worth 8 marks.